Manual de integração

Quéops Pirâmides ↔ UNO ERP

Documento técnico para a equipe de desenvolvimento do UNO ERP. Descreve a API pública da loja (v1), os webhooks, o modelo de sincronização bidirecional e o que ainda precisa ser construído de cada lado.

Versão 3.4 — 14 de outubro de 2026

Ambiente de produção: https://queopspiramides.com.br

1. O que existe hoje

A loja é uma aplicação Node/Express com MySQL e expõe uma API REST versionada (`/api/v1`) mais um mecanismo de webhooks. Leitura e **gravação** de produto já funcionam, com a trava por campo que preserva a autonomia do painel — quatro capacidades que a sincronização bidirecional exige ainda não existem, e estão listadas na seção 7 com o contrato proposto. Elas são pequenas do lado da loja; o que não dá é descobri-las no meio da implementação.

Capacidade	Situação	Onde
Ler catálogo (produtos ativos, preço, estoque, peso)	**Pronto**	`GET /products`
Ler pedidos com itens, valores e status	**Pronto**	`GET /orders`
Ler clientes com total gasto	**Pronto**	`GET /customers`
Atualizar estoque de um produto	**Pronto**	`PATCH /products/:id/stock`
Mudar status do pedido (faturado, enviado)	**Pronto**	`PATCH /orders/:id`
Ser avisado de pedido novo por webhook	**Pronto**	`order.created`
Gravar produto (preço, nome, peso, situação…)	**Pronto**	`PUT /products/:id`
Criar produto novo pela API	**Pronto**	`PUT /products/:id`
Enviar até 200 produtos numa chamada	**Pronto**	`POST /products/batch`
Trava por campo: o que o painel edita o ERP não sobrescreve	**Pronto**	`seção 6`
Buscar só o que mudou desde X (sincronização incremental)	A construir	`seção 7.1`
Endereço de entrega e transportadora no pedido	**Pronto**	`GET /orders`
Enviar a NF-e do pedido (XML + DANFE), com o cliente baixando na conta dele	**Pronto**	`seção 4.6`
Produto com variações (grade) numa página só, agrupado no painel	**Pronto**	`seção 4.7`

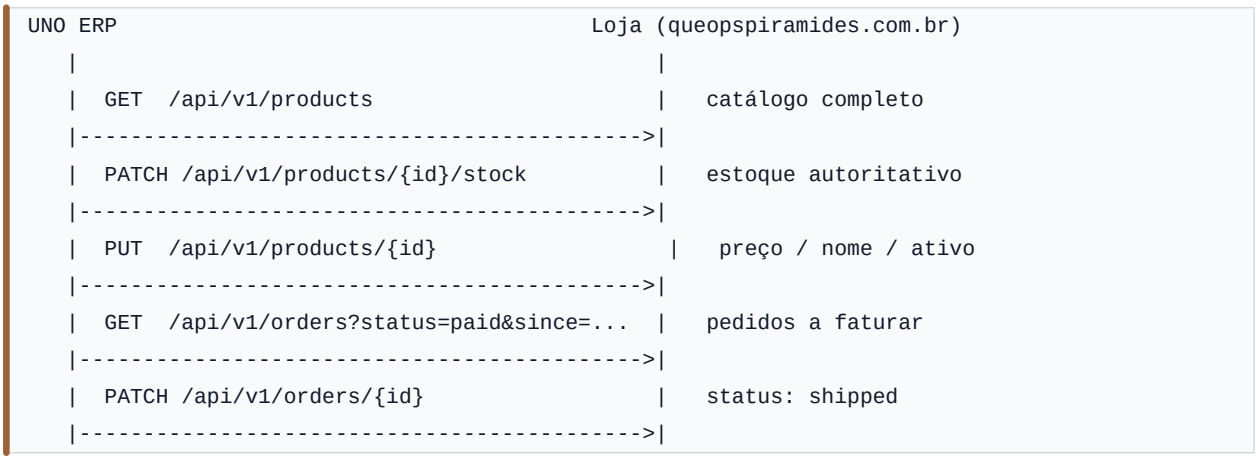
A regra de conflito já está implementada — leia a seção 6 antes de codificar

O ERP é a fonte da verdade de preço e estoque, e **o painel mantém autonomia**. Os dois requisitos convivem por meio da trava por campo: o que a lojista editar no painel passa a ser ignorado nos envios do ERP, e a resposta da gravação informa exatamente qual campo foi recusado (`ignored`). O ERP precisa ler esse campo — ignorá-lo faz a integração parecer instável, com valores que "não colam".

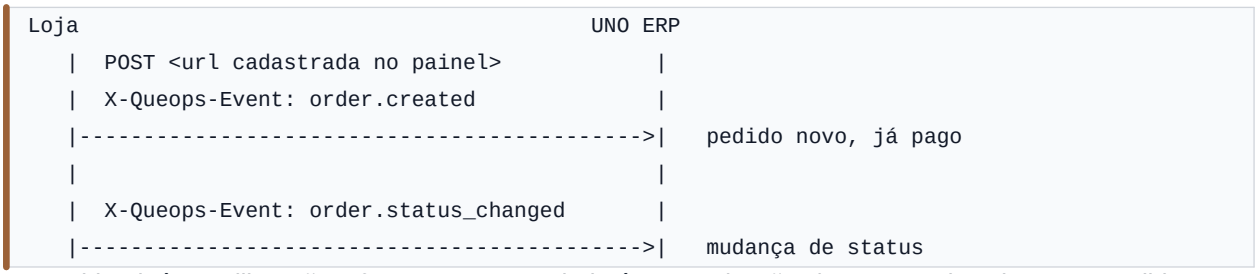
2. Arquitetura e fluxos

A integração tem dois sentidos, e eles usam caminhos diferentes de propósito: leitura e escrita partem do UNO (que controla o próprio ciclo de sincronização), e o aviso de evento parte da loja (que sabe no instante em que o pedido nasce).

2.1 UNO → loja (a cada ciclo de sincronização)



2.2 Loja → UNO (no instante do evento)



O webhook é o gatilho, não a fonte: o corpo enviado é enxuto (seção 5) e o UNO deve buscar o pedido completo em `GET /api/v1/orders/{id}` antes de faturar. Esse desenho é deliberado — é o mesmo princípio que a loja aplica com o Mercado Pago, onde o aviso diz apenas "algo mudou" e o valor verdadeiro vem sempre de uma consulta autenticada.

2.3 Cadência recomendada

Sincronização	Frequência	Chamada
Estoque (ERP → loja)	a cada 5–15 min, ou por evento de movimentação	`PATCH /products/:id/stock`
Preço (ERP → loja)	1× por dia, ou quando a tabela mudar	`PUT /products/:id · /batch`
Pedidos a faturar (loja → ERP)	webhook + varredura de segurança a cada 30 min	`GET /orders?status=paid&since=`
Catálogo completo	1× por dia (conferência)	`GET /products`

A varredura de segurança existe porque o webhook da loja é disparado uma única vez, sem repetição automática (seção 5.3). Um ERP que confie apenas no webhook perde o pedido cujo aviso falhou — e perder pedido pago é o pior defeito possível numa integração de ERP.

3. Autenticação

Toda chamada a `/api/v1` exige uma chave gerada no painel da loja, em **Integrações** → **API**. Não há cookie nem token CSRF: é comunicação servidor-a-servidor.

```
Authorization: Bearer qp_live_3f8a2c...          # 8 + 40 caracteres hexadecimais
Content-Type: application/json                   # nas chamadas com corpo
```

3.1 Ciclo de vida da chave

Característica	Comportamento
Formato	qp_live_ + 40 hex (20 bytes aleatórios)
Exibição	A chave inteira aparece **uma única vez**, na criação. Guarde no cofre do UNO.
Armazenamento na loja	Somente o hash (bcrypt). O prefixo de 16 caracteres é indexado para a busca.
Revogação	Imediata, pelo painel. Chamadas seguintes recebem 401.
Auditoria	A loja grava `last_used_at` a cada chamada válida.
Escopos	Não há: a chave dá acesso a todos os endpoints v1. Uma chave por integração.

3.2 Teste de credencial

```
curl -s https://queopspiramides.com.br/api/v1/products \
     -H "Authorization: Bearer qp_live_SEU_TOKEN" | head -c 300

# 200 → {"products":[{"id":"...","sku":"...","name":"...", ...}]}
# 401 → {"error":{"code":"invalid_api_key","message":"Envie uma chave válida ..."}}
```

4. Endpoints disponíveis

Base: `https://queopspiramides.com.br/api/v1`. Todas as respostas são JSON. Valores monetários vêm como número em reais (não centavos), com duas casas.

4.1 Categorias: código no ERP, slug na loja

Os dois sistemas identificam categoria de formas diferentes, e os dois têm razão para isso. O ERP usa **código**, porque nome muda. A loja usa **slug**, porque ele está na URL pública (`/?categoria=piramides`) e no sitemap já entregue ao Google — trocá-lo por `/?categoria=0012` quebraria o que está indexado, para resolver um problema que é de integração.

Então os dois convivem: o ERP manda a lista dele, alguém diz na loja onde cada código entra, e a partir daí produto vai e volta por código.

PUT `/categories` — **PRONTO**

Carga das categorias do ERP. Absoluta e idempotente: mande a lista inteira a cada ciclo.

```
PUT /api/v1/categories
{ "categories": [
    { "code": "0001", "name": "Pirâmides" },
    { "code": "0004", "name": "Pulseiras", "parentCode": "0003" },
    { "code": "0005", "name": "Uso Interno", "active": false } ] }

200 -> { "ok": true, "recebidas": 3, "criadas": 3, "atualizadas": 0,
         "pendentes": 3, "warnings": [],
         "message": "3 categoria(s) ainda sem destino na loja..." }
422 -> invalid_batch (sem lista) | batch_too_large (mais de 2000)
```

Item inválido não derruba o lote: ele vai para `warnings` e os outros são gravados. A carga **não apaga** o que sumiu do lote — uma carga truncada por timeout apagaria categorias vivas e, com elas, a amarração feita à mão. Para aposentar uma categoria, mande `active: false`.

A amarração é MANUAL, e é ela que libera o produto na vitrine

A loja não casa categoria por nome. Casar "Cristais" do ERP com "cristais" da loja parece óbvio, mas o mesmo palpite erra em "Pulseiras" quando existem duas, e o resultado é produto na seção errada da vitrine sem erro nenhum aparecer.

Enquanto um código estiver pendente, produto enviado com ele é **aceito e gravado** — a integração não trava —, mas fica **sem categoria e fora da vitrine**, e a resposta do PUT do produto diz isso em `warnings`.

Você NÃO precisa reenviar o produto depois. A loja guarda qual código cada produto estava esperando; no instante em que alguém amarra, os produtos represados entram na vitrine sozinhos, e a resposta da amarração diz quantos foram (`released`).

Fluxo síncrono: uma categoria por produto

Se o ERP prefere garantir a ordem — mandar a categoria do produto imediatamente antes do produto, e guardar no banco dele que aquela categoria já foi integrada —, use o PUT de categoria única abaixo. É o mesmo efeito da carga em lote, para um item.

PUT `/categories/{code}` — **PRONTO**

Uma categoria só. Para mandar a categoria logo antes do produto.

```
PUT /api/v1/categories/0004
{ "name": "Pulseiras", "parentCode": "0003" }

200 -> { "ok": true, "code": "0004", "name": "Pulseiras",
         "created": true,
         "linked": false,              // <- é ISTO que o cache deve guardar
         "category": null, "subcategory": null,
         "message": "Categoria registrada, mas ainda SEM destino na loja..." }
422 -> invalid_category   // sem "name"
```

Guarde "linked", não o 200

O 200 aqui significa "a loja registrou a categoria" — **não** significa "o produto vai aparecer na loja". Entre uma coisa e outra existe uma decisão humana: alguém precisa dizer em que categoria da vitrine aquele código entra.

Um ERP que grave "categoria 0004 integrada" ao ver o 200 e nunca mais toque no assunto vai concluir que está tudo certo enquanto os produtos dela estão fora da vitrine. Guardando `linked`, o próprio ERP sabe quais códigos ainda dependem da loja e pode reconsultar de vez em quando com o GET abaixo.

Vale dizer: mesmo no pior caso, nada se perde. Os produtos entram sozinhos na hora da amarração, sem reenvio.

Modo espelho. A loja pode, a pedido do dono, substituir a taxonomia dela por uma cópia exata da do ERP — cada categoria vira uma categoria da loja, cada filha vira subcategoria, e todos os códigos ficam amarrados de uma vez. Quando isso é feito, `linked` passa a ser verdadeiro para tudo e não sobra pendência. Não muda nada no que o ERP envia: continua sendo `categoryCode` no produto.

Código que nunca veio como categoria

Existe um caso que não é "pendente de amarração" e que, por um tempo, ficou sem saída: o produto chega com um `categoryCode` que **nunca** foi enviado em nenhuma carga de categorias. A loja não consegue amarrar um código que não conhece, e o produto fica fora da vitrine sem que exista nada na tela para amarrar.

Como reconhecer isso do lado de vocês

A resposta do PUT do produto distingue os dois casos em `warnings`, e a diferença importa:

• *"…ainda não está amarrado a uma categoria da loja"* → o código existe aqui, falta a decisão humana. Nada a fazer do lado de vocês.
• *"…não veio em nenhuma carga de categorias"* → **o cadastro de categorias do ERP está incompleto** para aquele código. Mande `PUT /api/v1/categories/{code}` e o produto entra sozinho.

A loja passou a ter uma saída própria para não ficar refém disso: em Painel → Categorias do ERP, o dono vê a lista dos produtos parados **agrupada pelo motivo** e pode **registrar o código órfão** à mão, dando a ele um destino na vitrine. Esse registro é um provisório declarado: ele nasce com o próprio código como nome, e no instante em que a carga de categorias de vocês chegar, o nome é corrigido e **a amarração é preservada** — a carga nunca toca na amarração. Não é motivo para deixar de mandar a categoria.

Uma consequência prática para quem lê `productsWaiting`: o dono da loja também pode atribuir categoria a vários produtos de uma vez pelo painel. Quando isso acontece, eles deixam de estar esperando e o contador cai **sem** que nenhuma amarração de código tenha sido feita. O número continua respondendo a pergunta certa — "quantos produtos ainda dependem deste código" —, só não serve como registro histórico.

GET `/categories/{code}` **PRONTO**

O estado de um código só — consulta barata para conferir o cache.

```
200 -> { "code": "0004", "name": "Pulseiras", "parentCode": "0003",
         "active": true, "linked": true,
         "category": "acessorios", "subcategory": "pulseiras",
         "productsWaiting": 0 }      // produtos parados esperando este código
404 -> not_found   // código que a loja nunca recebeu
```

GET `/categories` **PRONTO**

A árvore da loja com os códigos amarrados, mais a lista do ERP com o estado de cada um.

```
200 -> {
  "categories": [                       // a árvore da LOJA
    { "id": "acessorios", "name": "Acessórios", "erpCode": "0003",
      "subcategories": [
        { "id": "pulseiras", "name": "Pulseiras", "erpCode": "0004" } ] } ],
  "erpCategories": [                    // o que o ERP mandou
    { "code": "0004", "name": "Pulseiras", "parentCode": "0003",
      "active": true, "category": "acessorios",
      "subcategory": "pulseiras", "linked": true } ],
  "pending": 2,                         // ativas ainda sem destino
  "productsWithoutCategory": 7          // produtos parados por isso
}
```

PUT `/categories/{code}/link` **PRONTO**

Amarra pela API, para quem já tem a correspondência pronta e não quer clicar.

```
PUT /api/v1/categories/0004/link
{ "category": "acessorios", "subcategory": "pulseiras" }

200 -> { "ok": true, "released": 12 }   // produtos que entraram na vitrine agora
422 -> invalid_link   // slug inexistente, ou subcategoria que não é dessa categoria
```

`{"category": null}` desamarra. Esta rota existe por conveniência; **o ciclo de sincronização não deve chamá-la** — se o ERP amarrasse sozinho, a decisão manual perderia o sentido.

4.2 Produtos

GET `/products` **PRONTO**

Catálogo completo, **apenas produtos ativos**, ordenados por posição e nome. Sem paginação: hoje devolve tudo numa resposta só.

```
{
  "products": [
    {
      "id": "piramide-cobre-15cm-1001",     // chave natural, usada em todos os endpoints
      "sku": "PIR-CO-15",                   // código interno; pode casar com o do ERP
      "name": "Pirâmide de Cobre 15cm",
      "category": "piramides",              // slug da loja (está na URL pública)
      "categoryCode": "0001",               // código no ERP, ou null se não amarrado
      "subcategory": "cobre",               // opcional
      "categoryLabel": "Pirâmides",
      "description": "texto curto",
      "longDescription": "texto completo",  // opcional
      "price": 189.9,
      "oldPrice": 219.9,                    // opcional (preço "de")
      "stock": 12,                          // número; aceita fração (12.5)
      "image": "/produtos/piramide-cobre-15cm.jpg",
      "weight": 1.2,                        // número, em QUILOS
      "weightLabel": "Base 15cm · cobre",   // só texto de vitrine; não é peso
      "tag": "mais vendido",                // opcional
      "highlight": true,                    // opcional
      "active": true,
      "pendingReview": false,               // retido p/ conferencia? (secao 4.7)
      "groupCode": null,                    // grupo de variacoes, ou null
      "groupName": null,
      "options": []                         // [{name,value,position}] se agrupado
    }
  ]
}
```

Os quatro últimos campos são **somente leitura** e entraram na 3.3 — `pendingReview`, `groupCode`, `groupName` e `options`. Eles contam o que a loja fez com o produto depois que ele chegou: se está retido para conferência e se pertence a um grupo de variações. Mandá-los numa gravação não tem efeito. A seção 4.7 explica os dois assuntos.

MUDOU NA 2.3: weight é número em quilos, e stock aceita fração

Até a versão 2.2, `weight` era texto livre ("1,2 kg") e `stock` recusava decimais. Os dois estavam errados, e a mudança **quebra quem lia weight como string**.

weight agora é número, em **QUILOS** — 0.2 são duzentos gramas. Enviar gramas por engano multiplica o frete por mil; a loja não recusa (peça de 150 kg existe), mas devolve aviso em `warnings` com a conta feita. Texto com unidade ("0,2kg") ainda é aceito, convertido e avisado — não conte com isso para sempre.

weightLabel é o antigo conteúdo de texto: medida da peça, exibida na vitrine. Não entra em cálculo nenhum. O ERP pode ignorá-lo.

stock aceita decimal, até 3 casas. Antes, 7,5 era recusado com 422; agora atravessa inteiro nos dois sentidos. Acima de 3 casas, arredonda e avisa.

Sem peso, o frete é cotado com 500 g por item

Peso ausente ou zero não dá erro: a cotação usa 500 g. Isso mantém a loja vendendo quando falta cadastro, e é a razão de o campo não ser obrigatório — mas para uma peça de 3 kg o frete cobrado do cliente sai bem abaixo do custo, e a diferença sai do bolso da loja sem aparecer em relatório nenhum. **Se o UNO é a fonte do peso, mandar peso é o que evita esse prejuízo.**

GET `/products/{id}` **PRONTO**

Um produto. Diferente da listagem, este endpoint devolve o produto **mesmo inativo** — útil para o ERP conferir um item que saiu da vitrine.

```
200 → { "product": { ...mesmos campos da listagem... } }
404 → { "error": { "code": "not_found", "message": "Produto não encontrado." } }
```

PUT `/products/{id}` **PRONTO**

Grava o produto: cria se não existir, atualiza se existir. **Aceita o mesmo formato da leitura** — o ERP pode ler um item, mudar um campo e devolver o objeto inteiro.

Só os campos presentes no corpo são tocados. Mandar `{"price": 199.9}` muda o preço e não zera o resto, o que permite ao ERP enviar apenas aquilo que ele controla.

```
PUT /api/v1/products/piramide-cobre-15cm-1001
{ "sku": "PIR-CO-15", "name": "Pirâmide de Cobre 15cm", "price": 199.9,
  "oldPrice": 229.9, "stock": 12.5, "weight": 1.2,
  "categoryCode": "0001", "active": true }

200 (ou 201 quando cria) →
{ "id": "piramide-cobre-15cm-1001",
  "ok": true,
  "criado": false,
  "applied": ["sku","name","price","oldPrice","stock","weight","category",...],
  "ignored": [ { "field": "price", "reason": "locked_in_panel" } ],
  "warnings": [ "campo \"preco\" não é gravável e foi ignorado" ] }
```

Campo da resposta	O que significa e o que fazer
`applied`	Campos gravados. Confira aqui, não no código HTTP.
`ignored`	Campos recusados por trava do painel (seção 6). Registre a divergência; reenviar não resolve.
`warnings`	Gravou, mas algo merece atenção: campo com nome errado, peso não reconhecido, categoria inexistente. **Trate como erro no seu lado** — cada aviso aqui é um dado que não chegou como você esperava.
`criado`	true quando o produto não existia. Útil para conciliar cadastro.

Campo com nome errado não dá erro — dá aviso

Enviar `"preco"` em vez de `"price"` devolve **200** com o aviso "campo não é gravável e foi ignorado", e o preço não muda. Foi uma escolha: recusar a requisição inteira por um campo extra quebraria integrações a cada campo novo que o ERP passasse a enviar. Em troca, **o ERP precisa ler** `warnings` — senão um erro de digitação vira preço desatualizado que ninguém percebe.

POST `/products/batch` **PRONTO**

Até 200 produtos numa chamada. Um item inválido não derruba os outros.

```
POST /api/v1/products/batch
{ "products": [ { "id": "...", "name": "...", "price": 25, "stock": 4.5 },
                { "id": "...", "price": 30 } ] }

200 → { "total": 2, "gravados": 2, "falhas": 0,
        "results": [ { "id": "...", "ok": true,  "applied": [...] },
                     { "id": "...", "ok": false, "error": { "code": "invalid_field",
                                                            "message": "..." } } ] }
422 → batch_too_large (mais de 200) · invalid_batch (lista ausente ou vazia)
```

A resposta é sempre 200 quando o lote foi entendido, mesmo com item recusado — `falhas` e o `results` de cada item contam o que aconteceu. Abortar o lote inteiro por causa de um produto faria o ERP reenviar 199 gravações que já tinham dado certo, e a repetição esconderia qual era o item ruim. Para 1.400 itens: 7 chamadas.

PATCH `/products/{id}/stock` **PRONTO**

Atalho para sincronizar só o estoque. Passa pela mesma regra de travas.

```
PATCH /api/v1/products/piramide-cobre-15cm-1001/stock
{ "stock": 7 }

200 → { "ok": true, "id": "piramide-cobre-15cm-1001", "stock": 7.5 }
422 → { "error": { "code": "invalid_stock", ... } }   // negativo ou não numérico
404 → { "error": { "code": "not_found", ... } }
```

```
PATCH /api/v1/products/piramide-cobre-15cm-1001/stock
{ "stock": 7.5 }

200 → { "ok": true, "id": "...", "stock": 7.5, "applied": ["stock"],
        "ignored": [], "warnings": [] }
404 → not_found      # este endpoint NÃO cria produto; use PUT para isso
```

O valor é absoluto, não incremental: mande o saldo, nunca a variação. Isso torna a chamada idempotente por natureza — repetir a mesma requisição não muda o resultado, o que importa quando a rede cai no meio de um ciclo. Se o estoque estiver travado no painel, vem em `ignored` e o saldo não muda.

GET `/orders` **PRONTO**

Pedidos, do mais recente para o mais antigo, com os itens embutidos.

Parâmetro	Valores	Observação
`status`	pending · paid · shipped · delivered · canceled	Valor fora da lista é ignorado (não dá erro)
`since`	data ISO 8601 — ex.: 2026-08-01T00:00:00Z	Compara com a CRIAÇÃO. Use na carga inicial
`updatedSince`	data ISO 8601	Compara com a ATUALIZAÇÃO. Use na varredura periódica

Na varredura de segurança, use updatedSince — não since

A varredura que cobre webhook perdido (seção 3) precisa encontrar o pedido que **mudou**, e não o que **nasceu**. Com `since`, um pedido criado ontem e pago hoje nunca reaparece: a criação continua sendo ontem. O pedido pago ficaria parado sem ninguém perceber — que é o pior defeito possível numa integração de pedido. `updatedSince` resolve isso, e a resposta vem ordenada pela atualização para que a paginação acompanhe o filtro.

```
GET /api/v1/orders?status=paid&updatedSince=2026-08-01T00:00:00Z

{
  "orders": [
    {
      "id": "QP-000142",                    // número do pedido na loja
      "createdAt": "2026-08-30T18:22:41.000Z",
      "updatedAt": "2026-08-30T19:05:12.000Z",  // filtro da varredura
      "customerId": "42",                   // cliente na loja, ou null
      "customerName": "Maria Oliveira",
      "customerEmail": "maria@exemplo.com",
      "customerPhone": "(11) 98888-7777",
      "customerDocument": "12345678909",    // SÓ DÍGITOS, ou null
```

```
  "customerDocumentType": "cpf",        // "cpf" | "cnpj" | null
  "customerCpf": "123.456.789-09",      // legado: como foi digitado
  "customerNote": "Entregar após as 18h",   // ou null
  "items": [
    { "productId": "piramide-cobre-15cm-1001",
      "sku": "1001",                    // é por aqui que o ERP casa
      "name": "Pirâmide de Cobre 15cm", // nome no momento da venda
      "quantity": 2,                    // aceita fração (peso, metro)
      "unitPrice": 189.9,               // preço no momento da venda
      "discount": 0,
      "totalPrice": 379.8 }
  ],
  "subtotal": 379.8,
  "shipping": 26.63,                    // frete COBRADO do cliente
  "shippingCostOwner": 26.63,           // custo assumido pela loja
  "discount": 18.99,                    // total, soma dos dois abaixo
  "discountCoupon": 18.99,
  "discountPayment": 0,
  "total": 387.44,
  "currency": "BRL",
  "couponCode": "BEMVINDO10",           // ou null
  "status": "paid",                     // esteira, como sempre
  "paymentStatus": "paid",              // pending|paid|refused|refunded
  "fulfillmentStatus": "unpacked",      // unpacked|shipped|delivered
  "cancelReason": null,
  "canceledBy": null,                   // customer|store|gateway|erp
  "payment": "pix",                     // "pix" | "card"
  "paymentDetails": {
    "method": "pix",
    "brand": null,                      // bandeira, quando cartão
    "installments": 1,
    "paidAmount": 387.44,               // null enquanto não pago
    "gateway": "mercadopago",
    "transactionId": "1234567890",
    "paidAt": "2026-08-30T19:05:12.000Z",
    "detail": null                      // motivo da recusa, quando houve
  },
  "paidAt": "2026-08-30T19:05:12.000Z",
  "shippedAt": null,
  "deliveredAt": null,
  "canceledAt": null,
  "channel": "site",
  "shippingAddress": {                  // para nota fiscal e etiqueta
    "cep": "44823-478", "street": "Rua das Flores", "number": "250",
    "complement": "Apto 42", "neighborhood": "Centro",
    "city": "Jacobina", "state": "BA",
    "recipientName": "Maria Oliveira",  // entrega para terceiro
    "phone": "(11) 98888-7777",
    "country": "BR",
    "cityIbgeCode": null                // a loja não coleta
  },
```

```
      "billingAddress": null,               // null = igual ao de entrega
      "shippingService": "Jadlog · .Package — até 5 dias úteis",
      "shippingCarrier": "Jadlog",          // case a transportadora POR AQUI
      "shippingServiceCode": "2",
      "shippingServiceName": "Jadlog · .Package",
      "shippingMinDays": 5,
      "shippingMaxDays": 5,
      "deliveryEta": "2026-09-08",          // ou null
      "trackingCode": null,                 // null quando ainda não há
      "trackingStatus": null,
      "trackingUrl": null
    }
  ]
}
```

Limite de 200 pedidos e ausência de paginação

A resposta é cortada em **200 pedidos**, sem cursor. Enquanto o volume diário estiver abaixo disso, sincronizar com `since` a cada 30 minutos resolve. Acima disso, um ciclo pode perder pedido em silêncio — a seção 7.2 propõe a paginação.

MUDANÇA DE CONTRATO: campos vazios agora são null

`trackingCode`, `trackingStatus` e `customerNote` saíam como `""` quando não havia valor. Agora saem como `null`, para que a pergunta seja "existe?" e não "está em branco?". **Quem já consome estes campos precisa ajustar a leitura.** `customerCpf` foi mantido como estava — continua vindo com máscara, e `""` quando ausente — justamente para não quebrar quem já o lê; para faturar, use `customerDocument`, que traz só dígitos.

Quando shippingCarrier vem null — e o que fazer

A transportadora sai da **cotação**. Com Correios ou Melhor Envio ligados na loja, `shippingCarrier`, `shippingServiceCode` e `shippingServiceName` vêm preenchidos com o que foi cotado. Quando a loja cobra frete pela **tabela dela** (preço por estado ou faixa de CEP), não houve cotação e não existe transportadora a declarar — a lojista pode informar uma transportadora padrão em Painel → Frete & Entrega, e se não informar os campos vêm `null`. **Null aqui significa "a loja não sabe", e não "a loja esqueceu": use o padrão do ERP.** Não derive a transportadora do texto de `shippingService` — ele é rótulo de vitrine e muda sem aviso.

Por que paymentStatus existe, se já há status

`status` é uma esteira linear: quando o pedido avança para `shipped`, a informação "foi pago" **desaparece** do campo, e não há como reconstruí-la. `paymentStatus` e `fulfillmentStatus` são os dois eixos separados; `status` continua existindo, com o mesmo significado de sempre. E `cancelReason` distingue pagamento recusado de desistência do cliente — no ERP, lançamentos diferentes.

O endereço é "shippingAddress", e não "shipping"

A versão 2.0 deste manual propunha o objeto sob o nome `shipping`. Ele foi implementado como `shippingAddress` por um motivo: `shipping` já existe nesta resposta como o **valor** do frete (número), e trocar o tipo de um campo publicado quebraria quem já consome a API. Um campo novo custa uma linha de documentação; um campo que muda de número para objeto custa uma integração parada.

GET `/orders/{id}` **PRONTO**

Um pedido com seus itens. É a chamada que o UNO deve fazer ao receber um webhook.

```
200 → { "order": { ...mesma estrutura da listagem... } }
404 → { "error": { "code": "not_found", "message": "Pedido não encontrado." } }
```

PATCH `/orders/{id}` **PRONTO**

Muda o status do pedido. É por aqui que o ERP confirma faturamento e expedição.

```
PATCH /api/v1/orders/QP-000142
{ "status": "shipped" }

# Ao cancelar, mande TAMBÉM o motivo — o ERP usa para separar
# desistência do cliente de pagamento recusado:
PATCH /api/v1/orders/QP-000142
{ "status": "canceled", "cancelReason": "Produto sem estoque" }

200 → { "ok": true }
422 → { "error": { "code": "invalid_status", "message": "Status inválido." } }
404 → { "error": { "code": "not_found", ... } }
```

Esta chamada **dispara o webhook** `order.status_changed` para todas as URLs cadastradas. Se o próprio UNO estiver inscrito nesse evento, ele receberá o aviso da mudança que ele mesmo fez — trate como eco e ignore, ou o ciclo se realimenta.

Status	Significado na loja	Efeito
`pending`	Pedido criado; pagamento não confirmado (Pix emitido, cartão em análise)	Estoque já reservado
`paid`	Pagamento confirmado pelo provedor	Liberado para faturar
`shipped`	Despachado	Nenhum
`delivered`	Entregue	Nenhum
`canceled`	Cancelado ou pagamento recusado	Estoque devolvido (uma única vez)

Não use PATCH para cancelar pedido pago

Mudar o status para `canceled` por esta rota **não** devolve estoque nem estorna cobrança — ela só grava o status. O cancelamento com devolução de estoque acontece no fluxo de pagamento da loja. Cancelamento originado no ERP deve, por ora, ser combinado por fora; a seção 7.3 propõe o endpoint que faz a coisa completa.

GET `/customers` **PRONTO**

Clientes com contagem de pedidos e total gasto. Limite de 500, sem paginação.

```
{
  "customers": [
    { "id": "42", "name": "Maria Oliveira", "email": "maria@exemplo.com",
      "phone": "(11) 98888-7777", "ordersCount": 3, "totalSpent": 1204.7,
      "createdAt": "2026-05-11T14:02:00.000Z" }
  ]
```

```
}
```

`totalSpent` soma os pedidos não cancelados.

> **CPF: liberado no pedido, não nesta listagem**
>
> A partir da versão 2.2 o CPF do comprador sai em `customerCpf` no **pedido** (seção 4.5), liberado a pedido do dono da loja para o UNO emitir NF-e ao consumidor. Aqui em `/customers` ele **não** sai: nota se emite contra a venda, não contra o cadastro. Vem vazio (`""`) quando o comprador não informou — nesse caso a nota é sem identificação do destinatário, e o UNO não deve tratar a ausência como erro de integração. **Consequência prática:** a chave da API v1 agora dá acesso a CPF de cliente. Quem tem a chave tem os CPFs — ela pertence ao cofre de credenciais do UNO, não a arquivo de configuração compartilhado nem a repositório, e o corpo destas respostas não deve ir para log.

4.6 Notas fiscais (NF-e)

O UNO emite a nota no faturamento e envia para a loja o **XML processado** e o **DANFE** de cada nota, amarrados ao pedido. Um pedido pode ter **várias** notas — faturamento parcial é o caso normal de quem tem estoque em mais de um lugar. O comprador passa a ver "Baixar nota fiscal" no pedido dele, e a lojista, no painel.

> **Esta é a única rota da API que NÃO fala JSON**
>
> O envio é `multipart/form-data`, com os binários como arquivos e não em base64: são até 3 MB somados, e base64 incharia 33% para depois obrigar os dois lados a montar a string inteira na memória. Todo o restante da v1 continua JSON.

POST `/orders/{id}/invoices` **PRONTO**

Sobe uma nota fiscal do pedido. Idempotente pela chave de acesso.

Campo	Tipo	Regra
`key`	texto, 44 dígitos	Chave de acesso da NF-e. É o identificador único da nota
`number`	inteiro	Número da nota
`series`	inteiro	Série
`issuedAt`	ISO 8601 **com fuso**	Data/hora da autorização na SEFAZ
`totalAmount`	decimal em texto	Valor total. Guardado como decimal exato, nunca float
`xml`	arquivo	XML processado, máximo de 1 MB
`pdf`	arquivo	DANFE, máximo de 2 MB

```
POST /api/v1/orders/QP-000142/invoices
Authorization: Bearer qp_live_...
Content-Type: multipart/form-data; boundary=...

key=35260912345678000199550010000123451000123456
number=12345   series=1
issuedAt=2026-09-22T14:32:10-03:00
totalAmount=1234.56
xml=@nota.xml   pdf=@danfe.pdf

201 (criada) ou 200 (mesma chave reenviada) →
{ "invoice": {
    "key": "3526...", "number": 12345, "series": 1,
    "issuedAt": "2026-09-22T14:32:10-03:00",
    "totalAmount": "1234.56", "status": "authorized",
    "xmlUrl": "https://.../api/v1/invoices/<token>.xml",
    "pdfUrl": "https://.../api/v1/invoices/<token>.pdf" } }
```

Reenvio é esperado. O UNO pode repetir a chamada à vontade: a mesma `key` substitui os arquivos e os metadados e responde `200`, sem criar duplicata. A garantia é uma **chave primária** no banco, e não uma checagem da aplicação — duas requisições simultâneas com a mesma chave não viram duas notas.

No reenvio, as URLs mudam. Tokens novos são sorteados e os arquivos anteriores são apagados: a URL antiga passa a responder 404, de propósito, porque ela já circulou por e-mail e por log. Guarde sempre as URLs da última resposta.

Esta rota NÃO muda o status do pedido

Faturar não é despachar. A transição para `shipped` continua exclusiva do `PATCH /orders/{id}`, e subir a nota não mexe em nada do pedido. Quem faz as duas coisas precisa fazer as duas chamadas.

PATCH `/orders/{id}/invoices/{key}` **PRONTO**

Cancela uma nota. Única transição aceita: authorized → canceled.

```
PATCH /api/v1/orders/QP-000142/invoices/3526...
{ "status": "canceled" }

200 → { "invoice": { ..., "status": "canceled" } }
422 → invalid_status     (qualquer valor que não seja "canceled")
404 → not_found          (a chave não é deste pedido)
```

Repetir o cancelamento de uma nota já cancelada responde `200` com a mesma nota — é *retry*, não erro. A nota cancelada **continua** aparecendo no GET do pedido com `status: "canceled"`, porque o UNO precisa do histórico completo; quem deixa de apresentá-la como válida é a loja, na conta do cliente e na rota de download.

GET `/orders/{id}` **PRONTO**

Passa a incluir "invoices": apenas URLs, nunca binários.

```
{ "order": { ..., "invoices": [ { "key": "...", "status": "authorized",
                                  "xmlUrl": "...", "pdfUrl": "..." } ] } }

# Pedido sem nota devolve lista VAZIA, não campo ausente.
```

É por aqui que o UNO confere o que já subiu, sem guardar estado próprio.

Privacidade dos arquivos

O XML da NF-e traz **CPF, nome e endereço** do comprador. Os arquivos ficam numa pasta que **não** é servida estaticamente (ao contrário de `/midia`, que é pública por ser vitrine) e saem por uma rota que confere quem está pedindo.

Quem baixa	Como se identifica	O que consegue
UNO	Chave de API no Authorization	Todas as notas, inclusive as canceladas
Lojista	Sessão do painel	Todas as notas, inclusive as canceladas
Comprador	Sessão da conta dele	Só as notas dos pedidos dele, e só as autorizadas
Qualquer outro	—	401 sem sessão; 404 com sessão de outra pessoa

O caminho do arquivo é um **token sorteado de 192 bits** e nada mais: não é a chave da NF-e, não é o id do pedido, não é derivado de nada. XML e PDF têm tokens diferentes, então vazar um não entrega o outro. As respostas vão com `X-Robots-Tag: noindex` e `Cache-Control: no-store`, e não há listagem de diretório.

Validação e erros desta rota

HTTP	code	Quando acontece
404	`not_found`	Pedido inexistente, ou chave inexistente no PATCH

HTTP	code	Quando acontece
422	`invalid_nfe_key`	A chave não tem exatamente 44 dígitos numéricos
422	`invalid_invoice`	Campo obrigatório ausente ou com tipo inválido
422	`invalid_status`	PATCH com status diferente de "canceled"
413	`file_too_large`	XML acima de 1 MB, PDF acima de 2 MB, ou corpo acima do teto
415	`unsupported_media_type`	O conteúdo não é PDF ou o XML não é bem formado
409	`invoice_key_conflict`	A chave já está registrada em OUTRO pedido
409	`invoice_canceled`	Reenvio de nota cancelada — ela não volta a valer

O tipo do arquivo é conferido pelo CONTEÚDO

O PDF precisa começar com `%PDF-` e o XML precisa ser bem formado — o `Content-Type` declarado não é levado em conta, porque quem o escreve é o cliente. Em compensação, o `application/octet-stream` que muitas bibliotecas HTTP mandam por padrão é aceito sem drama. **XML com** `<!DOCTYPE>` **ou** `<!ENTITY>` **é recusado**: NF-e autorizada nunca tem declaração de entidade, e é por ela que entra o XXE.

4.7 Produto com variações (grade)

A loja passou a ter um **produto pai**, montado no painel, que agrupa produtos já existentes no catálogo e os apresenta numa página só, com um seletor por eixo (Tamanho, Cor…). **Nada disso muda o contrato de integração.**

> **O agrupamento morre na vitrine**
>
> O UNO continua mandando **um produto por variação**, com `id` = `sku` = código do produto. O grupo **não é comprável**: não tem preço, não tem estoque, não tem SKU, não entra no carrinho e **não vira item de pedido**. Um pedido com duas variações do mesmo grupo é indistinguível de um pedido de dois produtos soltos.

Endpoint	Muda?
`POST /products/batch`	**Não**
`PUT /products/{id}`	**Não** — segue atualização parcial
`PATCH /products/{id}/stock`	**Não** — estoque continua por variação
`GET /products`	Forma **não** muda: lista plana, uma entrada por variação
`/orders/*, /categories/*, /invoices`	**Não**

Campos novos, somente leitura

O `GET /products` e o `GET /products/{id}` passam a trazer o agrupamento de cada produto. São **read-only**: se o UNO mandar estes campos numa gravação, eles são ignorados e a resposta avisa em `warnings`. Agrupar é decisão do painel — título, galeria e ordem dos seletores não existem no cadastro do ERP.

```
{
  "id": "PIR-COBRE-20",  "sku": "PIR-COBRE-20",
  "price": 240.00,       "stock": 2,
  "groupCode": "GRP-PIRAMIDE-DE-COBRE",   // null se nao agrupado
  "groupName": "Piramide de cobre",
  "options": [ { "name": "Tamanho", "value": "20cm", "position": 0 } ],
  "pendingReview": false                  // ver "produto novo", abaixo
}
```

`groupCode` vem `null` e `options` vem `[]` no produto sem grupo — e não ausentes: "o campo não veio" e "não está agrupado" precisam ser a mesma coisa do seu lado, senão uma loja desatualizada seria lida como catálogo todo agrupado.

Produto novo chega RETIDO

Esta é a única mudança de comportamento que o UNO sente. Produto **criado** pela API entra na loja com `pendingReview: true` e **não aparece na vitrine** até a lojista conferir, em Painel → Variações, se ele é uma peça solta ou a variação de um grupo.

> **O que isso significa, e o que NÃO significa**
>
> A gravação **funciona normalmente**: 201, o produto existe, aparece no `GET /products` e a sua reconciliação de vínculo o encontra. A resposta traz o aviso em `warnings`. **Atualização não retém**: produto que já estava publicado nunca sai da vitrine por causa de um envio seu. E a retenção é **desligável** pela lojista, em Painel → Configurações — quem vende só produto simples volta ao comportamento de antes.

O motivo é o card duplicado: quando existe um grupo e o UNO cria a variação nova, ela apareceria na vitrine como card solto **ao lado** do card do grupo — a mesma peça em dois lugares, sem erro em lugar nenhum, até alguém reparar.

Regras que a loja garante

Regra	Como a loja garante
O item do pedido carrega o SKU da VARIAÇÃO	O carrinho recusa o grupo: o botão do card abre a página para escolher
O mesmo SKU não está em dois grupos	O vínculo é uma coluna do produto, não uma tabela de ligação
Estoque é sempre por variação	O grupo não tem saldo; o seletor desabilita a opção esgotada
Agrupar não altera estoque, preço nem pedido	O grupo só guarda conteúdo de vitrine e o vínculo
Produto sem grupo se comporta como antes	Nenhum caminho novo passa por ele

Casos de borda, resolvidos: variação que o UNO desativa sai do seletor, e o grupo some da vitrine quando a última sair; grupo com **uma** variação renderiza como produto simples, sem seletor; combinação esgotada **aparece desabilitada**, não some; duas variações do mesmo grupo no carrinho são duas linhas; o card mostra o **menor** preço entre as variações com estoque; e a busca encontra o grupo pelo nome dele **ou por qualquer SKU** das variações.

5. Webhooks da loja

Cadastro em **Painel** → **Integrações** → **Webhooks**: uma URL por evento. A loja envia POST com JSON e o header `X-Queops-Event`.

5.1 order.created

```
POST <sua URL>
X-Queops-Event: order.created

{ "orderId": "QP-000142", "total": 387.44,
  "email": "maria@exemplo.com", "status": "aprovado" }
```

Atenção ao vocabulário de status deste evento

Aqui `status` traz o resultado da **cobrança** ("aprovado" ou "aguardando"), e não o status do pedido ("paid", "pending") usado em todo o resto da API. São dois vocabulários no mesmo nome de campo — uma inconsistência da loja, já registrada para correção. Até lá: não compare este campo com os status da seção 4; busque o pedido em `GET /orders/{id}` e use o status de lá.

5.2 order.status_changed

```
X-Queops-Event: order.status_changed

# quando a mudança vem do fluxo de pagamento:
{ "orderId": "QP-000142", "status": "paid", "detalhe": "accredited" }

# quando vem de PATCH /api/v1/orders/{id}:
{ "orderId": "QP-000142", "status": "shipped" }
```

O campo `detalhe` existe só no primeiro caso. Trate-o como opcional.

5.3 Entrega: o que a loja garante e o que não garante

Aspecto	Hoje
Tentativas	**Uma só.** Sem repetição automática, sem fila.
Timeout	5 segundos. Responda rápido e processe depois.
Resposta esperada	Qualquer 2xx. A loja não lê o corpo.
Assinatura	**Não há.** Não é possível provar que o POST veio da loja.
Ordem	Não garantida: dois eventos próximos podem chegar fora de ordem.
Duplicidade	Possível. Trate `orderId` + status como idempotente.

Consequência prática para o UNO

Enquanto não houver assinatura e repetição (seção 7.7), o webhook deve ser tratado como **dica**, não como verdade: ao recebê-lo, busque o pedido pela API antes de faturar. E mantenha a varredura periódica com `since` — é ela que recupera o aviso que se perdeu. Nunca aceite o corpo do webhook como ordem de faturamento: sem assinatura, qualquer pessoa que descubra a URL pode enviar um.

6. Fonte da verdade: ERP manda, painel mantém autonomia

A regra do cliente é clara e, tomada ao pé da letra, contraditória: **preço e estoque vêm do ERP**, mas **o painel precisa poder alterar quando necessário**. Sem um mecanismo explícito, o que acontece é o pior dos dois mundos — a lojista corrige um preço, o próximo ciclo do ERP o sobrescreve, e a conclusão natural é "o site está com defeito".

6.1 Como funciona a trava por campo (implementado)

Cada produto tem uma lista de campos travados. Quando alguém salva uma alteração no painel, **os campos que mudaram de valor** entram nessa lista e passam a ignorar o ERP. A comparação é por valor, não por presença: o painel envia o produto inteiro a cada salvamento, então tratar tudo como edição travaria o cadastro completo no primeiro clique — e o ERP nunca mais atualizaria nada.

```
# 1. a lojista muda o preço no painel: 189,90 → 149,90
#    → o campo "price" é travado automaticamente

# 2. o ERP manda preço e estoque no ciclo seguinte
PUT /api/v1/products/piramide-cobre-15cm-1001
{ "price": 210, "stock": 40 }

200 → { "applied": ["stock"],
        "ignored": [ { "field": "price", "reason": "locked_in_panel" } ] }

# 3. estado real: preço 149,90 (do painel) e estoque 40 (do ERP)

# 4. quando a promoção acabar, o painel solta a trava e o ERP volta a mandar:
#    Painel → Produtos → (produto) → "voltar a seguir o ERP"

# o produto também informa as travas na leitura, para o ERP saber antes:
GET /api/v1/products/{id} → { "product": { ..., "lockedFields": ["price"] } }
```

O ponto crítico do contrato: campo travado devolve **200 com o campo em** `ignored`, e não erro. Recusar a requisição inteira faria o ERP repetir para sempre um envio que nunca vai passar; devolver 200 mudo faria o ERP acreditar que aplicou. A terceira via — aceitar o que dá e relatar o que não deu — é a única que permite ao UNO registrar a divergência e seguir. **Do lado do UNO, isso precisa virar log ou alerta**: divergência silenciosa entre ERP e loja é o defeito que aparece no faturamento, semanas depois.

6.2 Matriz de responsabilidade sugerida

Campo	Dono	Observação
`stock`	ERP	Absoluto, a cada ciclo. Aceita fração. Travável no painel.
`categoryCode`	ERP	Código da categoria no ERP. Precisa estar amarrado (seção 4.1).
`category`	Loja	Slug — está na URL pública. O ERP lê, mas manda categoryCode.
`price / oldPrice`	ERP	Travável para promoção pontual da loja.
`sku`	ERP	Chave de conciliação; a loja não deve alterar.
`name / description`	ERP com sobrescrita	A loja costuma preferir o texto de vitrine ao do ERP.

Campo	Dono	Observação
`weight`	ERP	Número, em quilos. Alimenta o frete; vazio custa dinheiro (seção 4).
`weightLabel`	Painel	Medida exibida na vitrine. O ERP pode ignorar.
`image / longDescription / tag / highlight`	Loja	Conteúdo de vitrine; o ERP não deve enviar.
`active`	ERP com sobrescrita	ERP inativa por descontinuação; loja, por curadoria.
`status do pedido`	Compartilhado	Loja até paid; ERP de paid em diante.

7. O que precisa ser construído

Itens do lado da **loja**, com o contrato proposto para o UNO já poder programar contra ele. Nenhum depende do outro; podem ser feitos na ordem que a integração exigir.

7.1 Sincronização incremental do catálogo

Com ~1.400 itens, baixar o catálogo inteiro a cada ciclo é desperdício dos dois lados. Hoje a leitura não filtra por data e a listagem traz só produtos ativos.

```
GET /api/v1/products?since=2026-08-30T00:00:00Z&includeInactive=1
```

7.2 Paginação em /orders e /customers

Hoje o corte é 200 pedidos e 500 clientes, sem cursor — acima disso um ciclo perde registro em silêncio.

```
GET /api/v1/orders?limit=100&cursor=<opaco>
→ { "orders": [...], "nextCursor": "..." }      # ausente na última página
```

7.3 Cancelamento completo a partir do ERP

Mudar o status para `canceled` pela rota atual só grava o status: não devolve estoque nem estorna. O endpoint abaixo faria a coisa completa.

```
POST /api/v1/orders/{id}/cancel
{ "reason": "ruptura de estoque" }
→ devolve estoque (uma única vez) e registra o motivo
```

7.4 Assinatura e repetição dos webhooks

Assinatura HMAC-SHA256 do corpo com segredo compartilhado, no padrão que a loja já usa para o Mercado Pago, mais três tentativas com espera crescente.

```
X-Queops-Event: order.created
X-Queops-Signature: t=1787061681,v1=<hmac_sha256(segredo, "t.corpo")>
```

8. Erros

Todo erro sai no mesmo formato, e o `code` é estável — trate por ele, nunca pela mensagem, que é escrita para gente e muda.

```
{ "error": { "code": "invalid_stock", "message": "Informe \"stock\" como um inteiro não negativo." } }
```

HTTP	code	Quando acontece	O que o UNO deve fazer
401	`invalid_api_key`	Chave ausente, errada ou revogada	Parar o ciclo e alertar; não repetir
404	`not_found`	Produto ou pedido inexistente	Registrar divergência de cadastro
422	`invalid_stock`	stock negativo ou não inteiro	Corrigir o dado na origem
422	`invalid_status`	Status fora da lista permitida	Corrigir o mapeamento
422	`invalid_field`	Valor recusado na gravação (preço negativo, nome vazio…)	Corrigir o dado; a mensagem nomeia o campo
422	`missing_name`	Produto novo sem "name"	Enviar o nome, ou conferir o id
422	`invalid_batch`	Lote sem "products" ou vazio	Corrigir o corpo
422	`batch_too_large`	Mais de 200 produtos no lote	Dividir em lotes de 200
422	`invalid_nfe_key`	Chave de NF-e sem 44 dígitos	Corrigir a chave na origem
409	`invoice_key_conflict`	Chave de NF-e já usada em outro pedido	Conferir a que pedido a nota pertence
409	`invoice_canceled`	Reenvio de nota cancelada	Emitir nota nova; cancelada não reativa
413	`file_too_large`	XML acima de 1 MB ou PDF acima de 2 MB	Reduzir o arquivo; não repetir igual
415	`unsupported_media_type`	Conteúdo não é PDF, ou XML mal formado	Conferir o arquivo antes de reenviar
500	`schema_outdated`	Banco da loja desatualizado após um deploy	Avisar o responsável: falta rodar a migração
500	`internal_error`	Falha inesperada	Repetir com espera crescente
503	`db_unavailable`	Banco fora do ar	Repetir com espera crescente

Não há limite de requisições (*rate limit*) nos endpoints v1 hoje. Isso não é convite: a loja e a API dividem o mesmo processo Node, então um laço agressivo do ERP degrada a vitrine para os clientes. Cadencie do lado do UNO.

9. Checklist de homologação

Sequência mínima para considerar a integração aprovada. Cada passo é verificável e tem um resultado esperado — se algum falhar, o problema está antes, não depois.

#	Passo	Resultado esperado
1	Gerar a chave no painel e guardá-la no cofre do UNO	A chave aparece uma única vez
2	GET /products com a chave	200 e o catálogo; 401 sem a chave
3	PATCH /products/{id}/stock com um valor conhecido	O painel mostra o novo estoque na hora
4	PATCH com stock negativo	422 invalid_stock (a loja recusa dado inválido)
5	PUT /products/{id} com um produto novo	201 e criado:true; o produto aparece no painel
6	PUT com um campo escrito errado ("preco")	200 com o aviso em warnings — e o preço NÃO muda
7	Editar o preço desse produto no painel e repetir o PUT	price vem em ignored; o valor do painel permanece
8	POST /products/batch com 3 itens, um inválido	200 com gravados:2, falhas:1 e o motivo do item ruim
9	Fazer uma compra de teste na loja	Webhook order.created chega na URL do UNO
10	Buscar o pedido por GET /orders/{id} a partir do aviso	Itens, valores e status conferem com a tela
10b	Conferir shippingAddress e shippingService nesse pedido	Endereço completo e transportadora escolhida pelo cliente
11	PATCH /orders/{id} para shipped	Painel mostra "enviado"; webhook order.status_changed dispara
12	Desligar a URL do webhook e fazer outra compra	A varredura por since encontra o pedido — prova o plano B
13	POST /orders/{id}/invoices com o XML e o DANFE de uma nota de teste	201; o pedido passa a trazer invoices e o status NÃO muda
13b	Repetir o mesmo POST, com a mesma key	200, sem duplicata — e as URLs anteriores param de responder
13c	Abrir a xmlUrl sem a chave de API	401; com a conta do comprador dono do pedido, 200
14	Criar um produto novo pela API	201 com pendingReview:true e o aviso da retenção em warnings
14b	Procurá-lo no GET /products	Aparece normalmente — a reconciliação de vínculo não é afetada
15	Agrupar duas variações no painel e ler o GET /products	Lista continua plana; cada uma traz groupCode, groupName e options
15b	PUT /products/{id} mandando groupCode	200, o campo é IGNORADO e o aviso vem em warnings
15c	Comprar duas variações do mesmo grupo e ler GET /orders/{id}	Duas linhas, com o SKU de cada variação; o código do grupo em lugar nenhum
16	Revogar a chave no painel e repetir o passo 2	401 imediato

9.1 Contato técnico e ambiente

Item	Valor
Produção	https://queopspiramides.com.br/api/v1
Homologação	Não há ambiente separado. Combine uma janela e use produtos de teste inativos — assim nada aparece na vitrine.
Painel da loja	https://queopspiramides.com.br/admin → Integrações → API
Responsável técnico	Marcelo Oliveira — marcelo.oliveira@unosolucoes.com.br
Stack da loja	Node 20+ · Express · MySQL 8 / MariaDB · React

Este manual descreve o estado do código na data da capa. As seções 1 e 7 são as que mudam quando a loja evoluir — confira a versão antes de começar uma implementação nova.